

What Is Your Moodle Doing?

This presentation shows you how to send Moodle logs and timing data to OpenSearch using OpenTelemetry. In OpenSearch you can run searches to analyse the state of your Moodle site.

The slides and the demo files are at <https://github.com/netsapiensis/MoodleMootNordic2026>, in the folder Moodle-Otel-Opensearch.

Agenda

- 1 The problem with the Moodle log table
- 2 What Moodle 5.2, OpenTelemetry and OpenSearch each do
- 3 How the three fit together, and where each component runs
- 4 Terms used in the commands and in the searches
- 5 Installation and configuration on Ubuntu and Red Hat
- 6 Searches and charts in OpenSearch
- 7 Plan for personal data in the events and for too many indices
- 8 A short look at timing data, and then a live system

Moodle already logs every event

```
"Login failed for user 'admin'. Most likely the  
password did not match (error ID '3')."
```

```
"The user with id '2' viewed the 'assign' activity  
with course module id '2'."
```

- Every click, every login and every failure becomes one row in `mdl_logstore_standard_log`. That table is in the same database that serves your pages, and it is usually one of the **largest tables you have**.
- Moodle's built-in reports, such as **Logs** and **Live logs**, read that table directly.
- By default Moodle never deletes these rows, so the table grows for as long as the site runs.

Questions the core Moodle reports cannot answer

- How many **logins per hour** were there this week and last week?
- Which **courses are actually used**, and which has nobody opened since term start?
- What did **this one user** do, across every course, last month?
- Are **failed logins** spiking right now?

In OpenSearch you can answer each of these questions. One more question is why a page was slow. That question gets a short look at the end of this presentation.

Moodle produces two kinds of records: events and timings

An event records what happened in Moodle

user_login_failed
course_section_created
enrol_instance_created
capability_assigned

A timing records how long one run took (a short look at the end)

every web request · every CLI run · every scheduled task · every
adhoc task · every web service call

OpenTelemetry sends the Moodle data to OpenSearch

OpenTelemetry is a log shipper. It is distributed as open source. It takes the data that Moodle produces, the events and the timings, and sends it to OpenSearch.

There is no package called OpenTelemetry. It is a family of pieces. These four appear in this talk:

<code>ext-opentelemetry</code>	a PHP extension — hooks into running PHP code without editing it
<code>SDK</code>	PHP libraries — turn the Moodle data into records, in memory
<code>exporter</code>	a PHP library — encodes the records and sends them over the network
<code>OpenTelemetry Collector</code>	a separate service — receives everything, filters it, forwards it to storage

None of them stores data or displays it. That is OpenSearch's job.

<https://opentelemetry.io>

OpenSearch stores the events and searches them

OpenSearch is an open-source search engine and document store, forked from **Elasticsearch**, with a web front end called **OpenSearch Dashboards**.

What it does

Indexes every event, and every timing, as one document, then answers questions about millions of them in milliseconds. OpenSearch Dashboards draws charts from them. **Indexing** means every field — duration, task name, file path, line number — is written into a structure built for searching.

Why choose OpenSearch

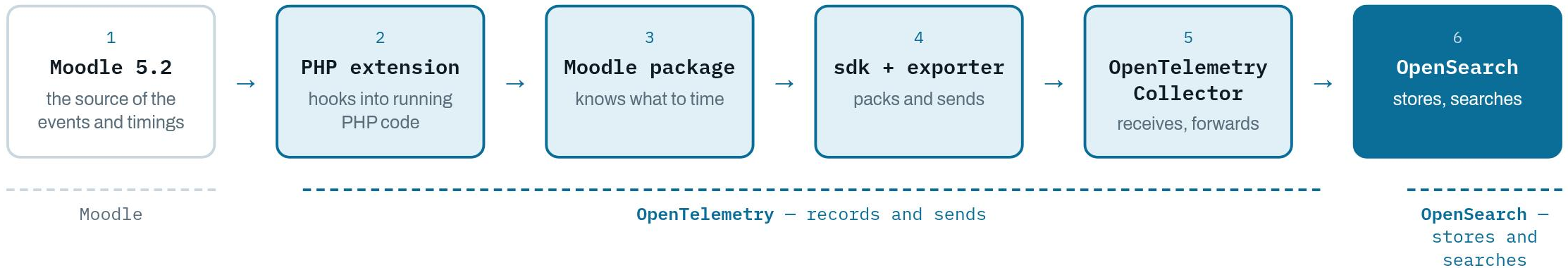
- **Grows by adding servers.** An index is split into shards spread over the nodes of a cluster; a replica of each shard survives a lost node.
- **Speaks OpenTelemetry.** The OpenTelemetry Collector ships an OpenSearch exporter; OpenSearch Dashboards has Trace analytics built in.
- **Fast on large volumes.** A search looks up the index instead of scanning every stored document, like the index at the back of a book, so it stays fast as the data grows.
- **Retention built in.** A policy deletes whole indices on schedule.

<https://opensearch.org>

How do events travel from Moodle to OpenSearch?

Why you need both OpenTelemetry and OpenSearch

Moodle produces the data. OpenTelemetry records it and sends it. OpenSearch keeps it and searches it.



Four of the six components are OpenTelemetry — and **none of them ships with Moodle**: you install all four separately. Moodle 5.2's only contribution is that it loads them once they are in place.

Where does each component run?



OTLP is OpenTelemetry's network protocol: the format of every record sent between components. Port 4318.

OTEL_EXPORTER_OTLP_ENDPOINT is an OpenTelemetry environment variable, read by components 2 to 4, not a Moodle setting. It goes in the PHP-FPM pool file and the cron file.

**These terms appear in the
commands and in the searches**

Log record: one Moodle event stored as one document

When anything happens in Moodle, it emits an **event**. Shipped to OpenSearch, each event becomes one document.

```
body      "The user with id '2' viewed the 'assign' activity  
          with course module id '2'."  
time      2026-08-23T21:30:58Z  
service    moodlelms
```

Attributes: the key = value pairs on a log record

Every record carries a list of **key = value** pairs, its attributes. OpenSearch stores each one as a searchable field under the prefix `attributes.` This is the same log record with seven of its seventeen attributes. The body sentence is built from the first three.

```
body      "The user with id '2' viewed the 'assign' activity
           with course module id '2'."
time      2026-08-23T21:30:58Z
service   moodlelms

attributes.event.data.userid      2          # the user with id '2'
attributes.event.data.objecttable assign      # the 'assign' activity
attributes.event.data.contextinstanceid 2          # course module id '2'
attributes.event.data.action      viewed
attributes.event.data.eventname    \mod_assign\event\course_module_viewed
attributes.event.data.crud         r
attributes.event.data.courseid     2
```

We install and configure the components in nine steps

COMPONENT 2 · THE PHP EXTENSION

1. Install the C compiler, the PHP header files and pecl
2. Install and enable the OpenTelemetry PHP extension

COMPONENTS 3 AND 4 · THE MOODLE PACKAGE, THE SDK AND THE EXPORTER

3. Install PHP Composer and the two OpenTelemetry PHP packages
4. These six environment variables configure the OpenTelemetry PHP libraries
5. Write the six environment variables into two files

COMPONENT 5 · THE OPENTELEMETRY COLLECTOR

6. Install the OpenTelemetry Collector
8. Configure the OpenTelemetry Collector to receive and send the records

COMPONENT 6 · OPENSEARCH

7. Configure OpenSearch

THE CHECK

9. Check with curl that the events have arrived in OpenSearch

Install the C compiler, the PHP header files and pecl

To install the OpenTelemetry PHP extension, the server needs the PHP header files, the `pecl` command and a C compiler. The package `php-pear` provides the `pecl` command.

Ubuntu 24.04 LTS

```
apt install php8.3-dev php-pear \  
gcc make autoconf
```

RHEL 9 • Rocky 9

```
dnf install php-devel php-pear \  
gcc make autoconf
```

Install and enable the OpenTelemetry PHP extension

PECL is the PHP site that publishes extensions. The command `pecl install opentelemetry` downloads the extension and builds the file `opentelemetry.so` for the PHP version on your server. A small `php.ini` file then tells PHP to load it.

Ubuntu 24.04 LTS

```
pecl install opentelemetry    # compiles the C source with gcc

echo "extension=opentelemetry.so" \
> /etc/php/8.3/mods-available/opentelemetry.ini

phpenmod opentelemetry
systemctl restart php8.3-fpm
```

RHEL 9 • Rocky 9

```
pecl install opentelemetry    # same: compiles with gcc

echo "extension=opentelemetry.so" \
> /etc/php.d/40-opentelemetry.ini

# no phpenmod – the drop-in is enough
systemctl restart php-fpm
```

Run `php --ri opentelemetry` to check the result. The output must contain **opentelemetry hooks => enabled**.

Install PHP Composer and the two OpenTelemetry PHP packages

The OpenTelemetry PHP code comes as two packages. PHP Composer is the tool that downloads PHP packages. Install PHP Composer first.

Ubuntu 24.04 LTS

```
# packaged, version 2.7  
apt install composer
```

RHEL 9 • Rocky 9

```
# not in AppStream – use the installer  
curl -sS https://getcomposer.org/installer \  
| php  
mv composer.phar /usr/local/bin/composer
```

Then run these two commands in the Moodle folder. They are the same on Ubuntu and on Red Hat. Each command downloads one package into the folder `/var/www/moodle/vendor/`.

```
cd /var/www/moodle  
composer require moodlehq/moodle-package-otel # records the Moodle events and timings  
composer require open-telemetry/exporter-otlp # sends the records to the OpenTelemetry Collector
```

These six environment variables configure the OpenTelemetry PHP libraries

The OpenTelemetry PHP code reads its settings from environment variables of the PHP process. These are the six.

```
OTEL_PHP_AUTOLOAD_ENABLED=true           # switches the OpenTelemetry PHP code on
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 # the address of the OpenTelemetry Collector
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf # the data format on the network
OTEL_TRACES_EXPORTER=otlp                 # sends the timings
OTEL_LOGS_EXPORTER=otlp                   # sends the events
OTEL_METRICS_EXPORTER=none                 # sends no metrics
```

Moodle has no setting for these variables. They do not go into `config.php`, and the Moodle admin pages do not show them.

Write the six environment variables into two files

Moodle runs PHP in two places that matter here. PHP-FPM runs it for web requests, and cron runs it for the scheduled tasks. Each has its own environment, so the same six lines go into two files.

1 · For web requests, add the lines to the PHP-FPM pool file.

Ubuntu `/etc/php/8.3/fpm/pool.d/www.conf` · RHEL `/etc/php-fpm.d/www.conf`

```
env[OTEL_PHP_AUTOLOAD_ENABLED] = true
env[OTEL_EXPORTER_OTLP_ENDPOINT] = http://localhost:4318
# add one env[] line for each of the six variables, then restart PHP-FPM
```

2 · For Moodle cron, add the lines to the cron file that you already have, above the schedule line.

`/etc/cron.d/moodle`

```
OTEL_PHP_AUTOLOAD_ENABLED=true
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
* * * * * www-data php /var/www/moodle/admin/cli/cron.php >/dev/null # RHEL: apache
```

Install the OpenTelemetry Collector

The OpenTelemetry Collector runs as a service. It receives the records from Moodle and sends them to OpenSearch. These steps install it on the Moodle server, and it can also run on its own host. It is not in the standard Ubuntu and Red Hat repositories, so you download the .deb or .rpm package from GitHub. The releases page on GitHub shows the newest version.

Ubuntu 24.04 LTS

```
V=0.161.0 # the version of the OpenTelemetry Collector
wget https://github.com/open-telemetry/\
opentelemetry-collector-releases/releases/\
download/v$V/otelcol-contrib_${V}_linux_amd64.deb

dpkg -i otelcol-contrib_${V}_linux_amd64.deb
```

RHEL 9 • Rocky 9

```
V=0.161.0 # the version of the OpenTelemetry Collector
wget https://github.com/open-telemetry/\
opentelemetry-collector-releases/releases/\
download/v$V/otelcol-contrib_${V}_linux_amd64.rpm

dnf install ./otelcol-contrib_${V}_linux_amd64.rpm
```

The package installs a systemd service with the name `otelcol-contrib` and the configuration file `/etc/otelcol-contrib/config.yaml`. The package starts the service, and the service listens on port 4318 for the records from Moodle.

Configure OpenSearch

OpenSearch needs four things before the first records arrive.

WHAT	WHERE YOU DO IT	WHY
A user for the OpenTelemetry Collector	OpenSearch Dashboards, Security, Internal users and Roles	The OpenTelemetry Collector logs in as this user. The role needs <code>create_index</code> and <code>write</code> on <code>otel-v1-logs-*</code> and <code>otel-v1-apm-span-*</code> , and the cluster permissions <code>cluster_manage_index_templates</code> and <code>indices:data/write/bulk</code> .
An index template	You do nothing. The OpenTelemetry Collector creates it at start.	The template sets the field types of each new monthly index. It needs OpenTelemetry Collector 0.157.0 or newer.
An index pattern	Dashboards Management, Index patterns, Create. The pattern is <code>otel-v1-logs-*</code> and the time field is <code>time</code> .	Discover is the search page of OpenSearch Dashboards. It shows no data without the pattern.
A retention policy	Index Management, State management policies, Create policy, with an ISM template for <code>otel-v1-logs-*</code> and <code>otel-v1-apm-span-*</code>	ISM (Index State Management) deletes old indices on a schedule. The policy covers only the indices that come after it.

Configure the OpenTelemetry Collector to receive the records

The PHP package `exporter-otlp` sends the records from Moodle to `http://localhost:4318`. In the file `/etc/otelcol-contrib/config.yaml`, you configure the OpenTelemetry Collector to listen on that port.

```
receivers:  # listens on port 4318 for the records from the PHP package
  otlp:
    protocols:
      http:
        endpoint: localhost:4318
processors:  # sends the records in groups, one group every 5 seconds
  batch:
    timeout: 5s
```

Configure the OpenTelemetry Collector to send the records to OpenSearch

The OpenTelemetry Collector needs the login, the address of OpenSearch and the names of the indices.

```
extensions:  # the user name and the password for OpenSearch
  basicauth/opensearch:
    client_auth:
      username: otelcol
      password: DemoCollector2026!
exporters:  # sends the records to OpenSearch
  opensearch/moodle:
    http:
      endpoint: https://opensearch1:9200  # the address of your OpenSearch server
      auth:
        authenticator: basicauth/opensearch
      tls:
        insecure_skip_verify: true  # does not check the certificate of OpenSearch
    logs_index: "otel-v1-logs-moodle"  # the name must start with otel-v1-logs
    logs_index_time_format: "yyyy.MM"  # one new index each month
    traces_index: "otel-v1-apm-span"  # the trace pages read this name, keep it
    traces_index_time_format: "yyyy.MM"
    mapping:
      mode: "otel-v1"
      manage_index_template: true  # creates the index template at start
service:  # nothing runs without this section, it switches the parts on
  extensions: [basicauth/opensearch]
  pipelines:
    logs: {receivers: [otlp], processors: [batch], exporters: [opensearch/moodle]}
    traces: {receivers: [otlp], processors: [batch], exporters: [opensearch/moodle]}
```

Restart the service with `systemctl restart otelcol-contrib`. The OpenTelemetry Collector reads the file only at start.

Check with curl that the events have arrived in OpenSearch

```
curl -sk -u admin "https://opensearch1:9200/otel-v1-logs-*/_count"  
{"count":3145,...}
```

The command asks for the admin password. The option `-k` tells curl not to check the certificate of OpenSearch.

If the count is 0, check these four places in this order, from OpenSearch back to PHP.

the index	<code>curl -sk -u admin "https://opensearch1:9200/_cat/indices/otel-★"</code>
the collector	<code>journalctl -u otelcol-contrib</code>
the variables	<code>grep OTEL_ /etc/php/8.3/fpm/pool.d/www.conf</code>
the extension	<code>php --ri opentelemetry</code>

**You can search the events in
OpenSearch and draw charts from them**

OpenSearch receives every Moodle event

Here are five examples of the events that Moodle records.

- Moodle records **every login**. For a failed login, the record also has the user name that was tried.
- Moodle records **every view of a course or an activity**. The record shows who viewed it.
- Moodle records **every create, update and delete**, such as an enrolment, a grade or an upload.
- Moodle records **every role change and every permission change**, and that gives you an audit trail.
- Moodle records **some failures**, such as an email that it could not send.

A search shows the hour when failed logins spiked

This search counts the failed logins in each hour.

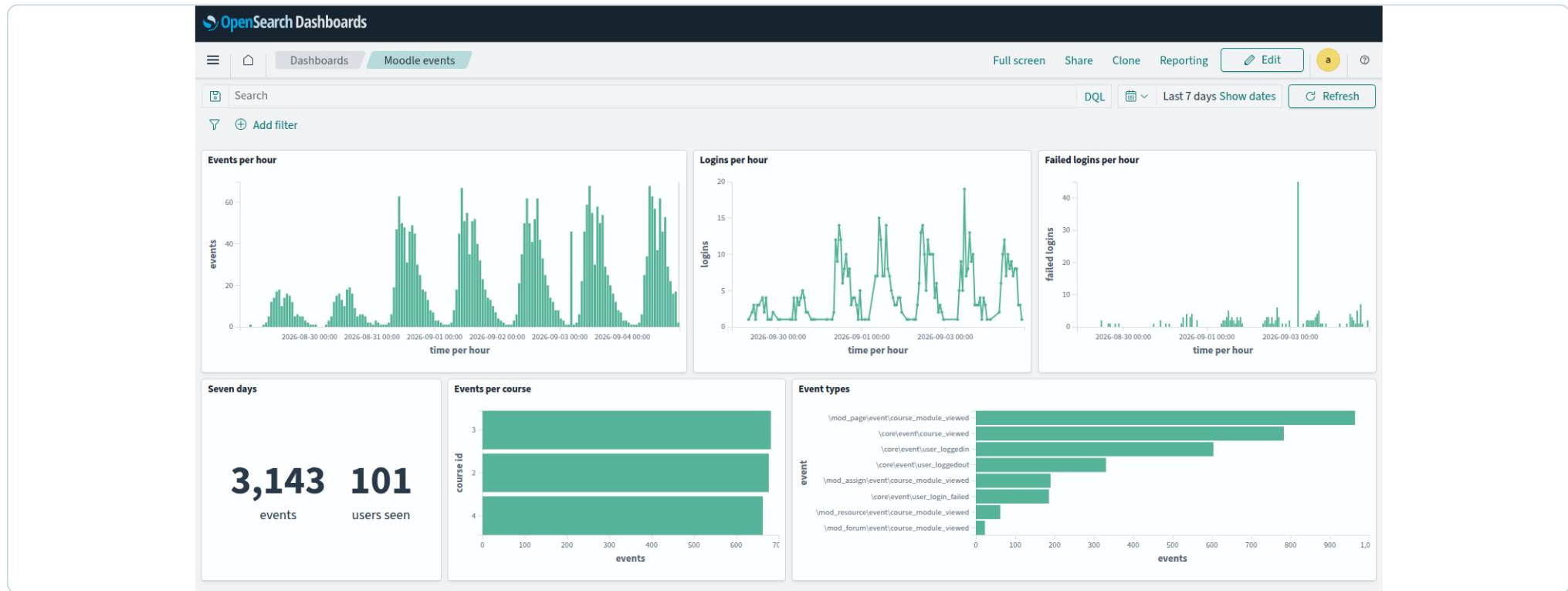
```
curl -sk -u admin "https://opensearch1:9200/otel-v1-logs-*/_search" -H 'Content-Type: application/json' \
-d '{"size":0,"query":{"term":{"attributes.event.data.eventname":"\\core\\event\\user_login_failed"}},'
  "aggs":{"per_hour":{"date_histogram":{"field":"time","fixed_interval":"1h"}}}}'
```

Here is part of the result from the demo site, as a table. The times are UTC.

2026-09-02 20:00	0
2026-09-02 21:00	2
2026-09-02 22:00	0
2026-09-02 23:00	0
2026-09-03 00:00	0
2026-09-03 01:00	0
2026-09-03 02:00	45
2026-09-03 03:00	0

OpenSearch Dashboards draws charts from the events

You build each chart once, and it updates when new events arrive. The picture shows six example charts with seven days of data from the demo site.



Plan for personal data in the events and for too many indices

Events can carry personal data

- By default an event identifies a user only **by id**, for example "the user with id '2'". A failed login also records the **user name that was typed**.
- Every email that Moodle fails to send is stored as an `email_failed` event, with the **full text of the email**. Moodle's own new-device sign-in notice is such an email, and it contains the user's name, email address, IP address and browser.
- The index holds personal data as soon as one record identifies a person. The General Data Protection Regulation (GDPR) then applies to the index, not only to Moodle.
- Remove or hash such attributes in the **OpenTelemetry Collector**, before they reach OpenSearch. For example, this statement in a `transform` processor deletes the email text:

```
delete_key(log.attributes, "event.data.other")
  where log.attributes["event.data.eventname"] == "\\core\\event\\email_failed"
```

- Decide the retention period **before** you switch the sending on.

OpenSearch stores every index in shards, and a server has a limit on shards

An index in OpenSearch is stored on disk as one or more pieces called shards. By default one data server accepts at most 1000 shards, and every new index counts against that limit.

What happens without a retention policy

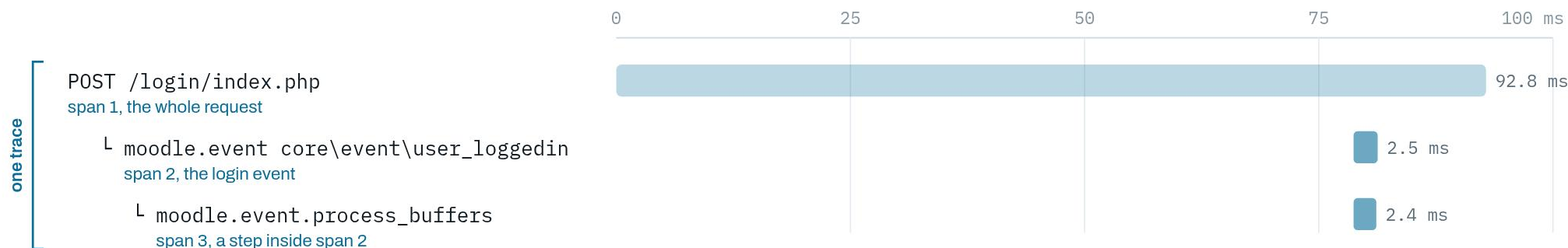
By default each index has two shards, one primary and one copy. For example, five daily indices add ten shards per day, and they reach 1000 shards in 100 days. From then on OpenSearch cannot create the new day's indices, and it rejects every incoming record. Nothing alerts you by default.

- Very large shards are also a problem. The OpenSearch documentation recommends 10 to 50 GB per shard. Above that, searches slow down.
- Choose the index period so that one index stays in that range. One index per month, `logs_index_time_format: "yyyy.MM"` in the OpenTelemetry Collector's configuration file, fits a site that writes up to a few million events a day.
- Create the retention policy in OpenSearch on the first day, so old indices are deleted and their shards are freed.

**Moodle also sends timing data
to OpenSearch through the
same OpenTelemetry Collector**

Every request also records how long each of its steps took

Every event record has two fields called `traceId` and `spanId`. They are there because the request that wrote the event also recorded how long each of its steps took. OpenTelemetry calls the timing of one request a **trace**. A trace is made of **spans**. A span is one step of the request, with a start time and a duration.



```
# part of the note on span 1, as stored in OpenSearch
moodle.shutdown_handler dbtime 0.03228 dbqueries 13/9
```

One trace with three spans, from a real login request on the demo site. The first span is the whole request. The other two spans are steps inside it. Moodle adds the note in the box to the first span. `dbtime` is the database time in seconds, here 32 ms. `dbqueries` is reads/writes, here 13 reads and 9 writes.

The timing data shows where Moodle spends its time

It answers these questions.

- Which pages are slow, and how slow?
- Which scheduled task ran for 40 minutes last night?
- How long did each Moodle event take inside a request?

A span has attributes, the same as an event. So you search spans the way you search events, and you can sort them by duration.

In OpenSearch Dashboards this data is usually viewed under **Observability**.

LIVE

Now we look at a live system

Start the demo environment on your computer with Docker Compose

The folder `Moodle-0tel-Opensearch/` at <https://github.com/netsapiensis/MoodleMootNordic2026> has the demo files. Your computer needs Docker Compose v2 and about 3 GB of free memory. One command starts every container.

```
cd Moodle-0tel-Opensearch/compose
docker compose up --build      # the first run builds the Moodle image: 5-10 minutes
```

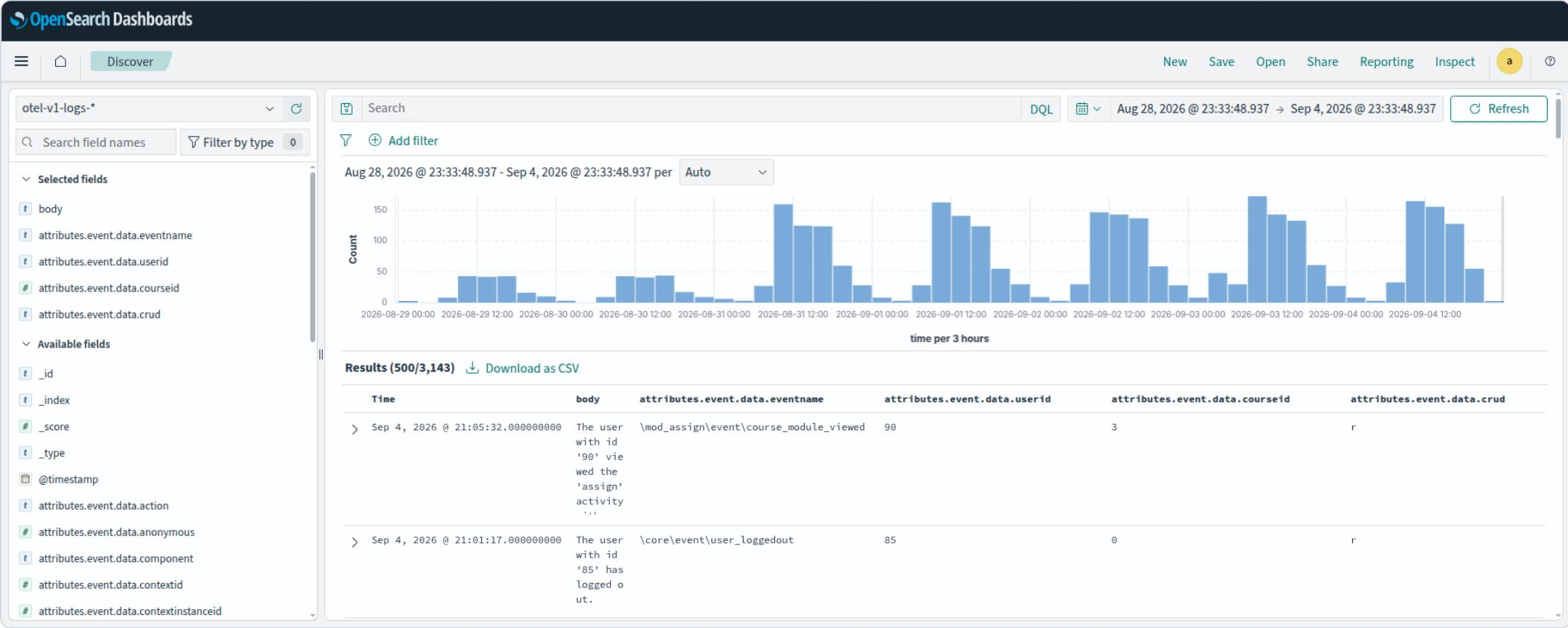
WHAT	WHERE	LOGIN
Moodle	https://localhost	admin / DemoAdmin2026!
OpenSearch Dashboards	https://localhost:5601	admin / DemoSearch2026!
OpenSearch	https://localhost:9200	admin / DemoSearch2026!

- The stack uses one self-signed certificate, so the browser warns once for each of the three addresses.
- The passwords are written as plain text in the files of the `compose/` folder, and the ports are open on your computer only.
- The stack is for a demo. Change the passwords and the certificate before other machines can reach it.

Docker Compose starts ten containers in this order

CONTAINER	IMAGE	WHAT IT DOES	RUNS
certs	alpine/openssl	Writes the self-signed certificate to <code>compose/certs/</code> on the first start.	once
mariadb	mariadb:11.4	Holds the Moodle database.	always
opensearch	opensearch:3.8.0	Stores and searches the events and the timing data. <code>https</code> on port 9200.	always
opensearch-init	opensearch:3.8.0	Creates the user and role the OpenTelemetry Collector logs in with.	once
opensearch-dashboards	opensearch-dashboards:3.8.0	Shows the screens Discover and Trace analytics. <code>https</code> on port 5601.	always
dashboards-init	opensearch:3.8.0	Loads the dashboard "Moodle events" with six charts into OpenSearch Dashboards.	once
otelcol	opentelemetry-collector-contrib:0.161.0	Runs the OpenTelemetry Collector, which receives the events and the timing data from Moodle and sends them to OpenSearch.	always
moodle-init	built from <code>docker/Dockerfile</code>	Installs the Moodle database on the first start.	once
moodle	the same built image	Runs Moodle 5.2 with the OpenTelemetry PHP extension and both PHP Composer packages. <code>https</code> on port 443.	always
moodle-cron	the same built image	Runs Moodle cron every minute, so the timing data of scheduled tasks exists too.	always

Discover shows one row for each Moodle event



The table shows the time, the body and four attributes of each event.

Read more at these six places

This talk	https://github.com/netsapiensis/MoodleMootNordic2026 , in the folder Moodle-Otel-Opensearch
Moodle	https://moodledev.io/docs/5.2/apis/telemetry
Package	https://github.com/moodlehq/moodle-package-otel
PHP extension	https://opentelemetry.io/docs/zero-code/php/
OpenSearch exporter	https://github.com/open-telemetry/opentelemetry-collector-contrib , folder exporter/opensearchexporter
OpenSearch	https://docs.opensearch.org — Discover, Index State Management, Trace Analytics

Thank you.